screenshot-mcp

MCP-сервер, который даёт агенту **настоящее окно консоли** на рабочем столе и **снимки экрана** штатным средством системы.

Ключевая сложность в том, что под Wayland не работают привычные приёмы автоматизации терминала: `xdotool` не вводит текст в нативные Wayland-приложения, `wmctrl` не существует, а `tmux` умеет управлять сессией, но не окном на экране. Здесь ввод идёт **прямо в псевдотерминал**, поэтому терминал показывает его так же, как ввод с клавиатуры, — без эмуляции клавиш и без каких-либо следов в интерфейсе.

Возможности:

- новое отдельное окно консоли с **твоим** `~/.bashrc` — твой промпт, цвета, алиасы;
- ввод команд и чтение вывода с кодом возврата;
- интерактивные программы: ввод, `Ctrl+C`, `Ctrl+D`, стрелки, история команд;
- снимок всего рабочего стола с поднятием окна консоли на передний план;
- снимок возвращается агенту прямо в ответе (картинкой), так что он может сам оценить, что попало в кадр.

Как это устроено

```
MCP-сервер (server.py)
   │  UNIX-сокет, JSON
   ▼
core/bridge.py — «насос»: переливает ввод и вывод
   │  псевдотерминал (pty)
   ▼
bash ── запущен прямо в окне терминала, наследует ~/.bashrc
```

- Настоящий `bash` работает **внутри** окна терминала и подхватывает пользовательский `.bashrc`. Заголовок окна при этом неотличим от обычного (`<каталог> : bash — Konsole`).
- Агент пишет байтами прямо в pty. Их обрабатывает tty-драйвер (отражение ввода) и `readline` — то есть это буквально тот же путь, что и при наборе с клавиатуры. Никакой эмуляции клавиш, поэтому в терминале не остаётся следов автоматизации.
- Код возврата берётся через хук `PROMPT_COMMAND`, который пишет маркер в служебный файл: на экране ничего не появляется.

Архитектура

Платформенные различия вынесены в бэкенды, ядро от них не зависит:

```
screenshot_mcp/
├── core/            pty-мост, сессия консоли, лаунчер — POSIX, работает везде
├── backends/
│   ├── terminal/    konsole · xterm · gnome-terminal
│   ├── screenshot/  spectacle · x11 · gnome
│   └── focus/       kwin · xdo_tool · atspi · none
├── config.py        определение окружения и пользовательские настройки
└── runtime.py       сборка окружения, настроек и бэкендов
```

Выбор бэкендов автоматический: определяется тип сессии (`XDG_SESSION_TYPE`) и рабочее окружение (`XDG_CURRENT_DESKTOP`). При необходимости его можно задать явно — см. Настройку.

Поддерживаемые окружения

Окружение	Окно терминала	Снимок экрана	Поднятие окна
KDE Plasma 6 / Wayland	Konsole	Spectacle	KWin-скриптинг
KDE Plasma / X11	Konsole, xterm	ImageMagick, scrot, maim	xdotool
GNOME / Wayland	gnome-terminal	gnome-screenshot	AT-SPI
X11 (любой DE)	xterm, gnome-terminal	ImageMagick, scrot, maim	xdotool

> **Что проверено вживую:** только вариант **KDE Plasma / Wayland** — на нём прогнаны все тесты. X11- и GNOME-пути написаны по документации используемых утилит (`scrot` , `maim` , `import` , `xdotool` , `gnome-screenshot` , AT-SPI) и требуют проверки на своём железе: терминал и снимок там работают через те же бэкенды ядра, но поднятие окна в X11 завязано на `xdotool` , а в GNOME — на AT-SPI, и доступность этих способов зависит от окружения.

Требования

- Python 3.11+ и пакет `mcp` ;
- терминал: `konsole` , `xterm` или `gnome-terminal` ;
- для снимков: `spectacle` (KDE) либо `scrot` / `maim` / `import` (X11) либо `gnome-screenshot` (GNOME);
- для поднятия окна: KWin (идёт с Plasma) либо `xdotool` на X11;
- `dbus-python` и `PyGObject` — для приёма отчётов от KWin-скриптов и AT-SPI.

Проверено на Debian 13, KDE Plasma 6.3, Wayland.

Установка

```
git clone git@github.com:Bra1nZ/screenshot-mcp.git
cd screenshot-mcp
pip install -r requirements.txt
```

Настройка

По умолчанию бэкенды выбираются по окружению. Если нужно указать явно, создай `~/.config/screenshot-mcp/config.toml`:

```
[terminal]
backend = "konsole"      # konsole | xterm | gnome-terminal

[screenshot]
backend = "spectacle"    # spectacle | x11 | gnome

[focus]
backend = "kwin"         # kwin | xdo_tool | atspi | none
```

Другой путь к конфигу можно задать переменной `SCREENSHOT_MCP_CONFIG`.

Подключение

opencode

`~/.config/opencode/opencode.json`:

```
{
  "mcp": {
    "desktop-console": {
      "command": ["python3", "/путь/к/screenshot-mcp/server.py"],
      "type": "local"
    }
  }
}
```

Другие MCP-клиенты

Сервер говорит на stdio, поэтому подходит любому клиенту с поддержкой локальных MCP-серверов. Команда запуска:

```
python3 /путь/к/screenshot-mcp/server.py
```

Сервер запускается **в графической сессии** и наследует `WAYLAND_DISPLAY` и `DBUS_SESSION_BUS_ADDRESS`; если переменных нет, подставляются значения по умолчанию для текущего пользователя.

Инструменты

Инструмент	Назначение
`console_open(cwd, cols, rows, title, terminal)`	открыть новое окно консоли, вернуть `console_id`
`console_exec(console_id, command, timeout, typing_delay_ms)`	напечатать команду и дождаться результата: `output`, `exit_code`, `timed_out`
`console_send(console_id, text, enter)`	напечатать текст без выполнения — для программ, ждущих ввода
`console_key(console_id, key)`	спецклавиши: `enter`, `tab`, `esc`, `up` / `down`, `ctrl+c`, `ctrl+d`, `f1` .. `f12`
`console_read(console_id, cursor)`	дочитать новый вывод
`console_state(console_id)` / `console_list()`	состояние консолей
`console_close(console_id)`	закрыть окно
`desktop_screenshot(path, console_id, delay_ms, preview)`	снимок рабочего стола; `console_id` — сначала поднять окно; возвращает метаданные и уменьшенную копию картинки
`desktop_focus(console_id)`	поднять окно консоли на передний план
`desktop_active_window()` / `desktop_list_windows()`	что сейчас на переднем плане, список окон

Пример сессии

```
console_open(cwd="~/projects/lab")        → console_id
console_exec(console_id, "make")          → output, exit_code
```

```
console_exec(console_id, "ls -la build/")  → убедиться, что артефакты на месте
desktop_screenshot(console_id=console_id)  → PNG в папке снимков + картинка в ответе
```

Ограничения

- **Экран должен быть разблокирован.** Пока активен lock screen, композитор не отдаёт снимки — это защита KWin, а не ошибка сервера. `desktop_screenshot` в этом случае вернёт понятное сообщение.
- Под GNOME поднять окно можно не всегда: там нет скриптинга композитора, а AT-SPI работает не у всех приложений. Если окно не поднялось, снимок всё равно делается — просто в кадре может быть другое окно.
- Символ табуляции в командах не поддерживается: `bash` воспринимает его как запрос автодополнения.
- `console_exec` ждёт нового промпта; для долгих задач нужен `timeout`, для интерактивных программ — `console_send` и `console_key`.

Безопасность

Сервер даёт подключённому агенту **полный контроль над консолью в графической сессии и чтение содержимого экрана**. Это инструмент автоматизации, а не средство изоляции: запускай его только с тем агентом, которому доверяешь, и не оставляй незакрытые окна консоли с активными сессиями.

Скриншоты складываются в стандартную папку для снимков и дублируются в `screenshots/` рядом с сервером — помни, что они могут содержать чувствительные данные с экрана.

Тесты

Тесты живые: они открывают настоящие окна и снимают настоящий экран, поэтому требуют запущенной графической сессии.

```
pip install -r requirements-dev.txt
python3 tests/test_backends.py      # юнит-тесты: конфиг и бэкенды (GUI не нужен)
python3 tests/test_bridge.py        # pty-мост: команды, вывод, код возврата
python3 tests/test_interactive.py   # ввод в программу, ctrl+c, история
python3 tests/test_e2e.py           # полный цикл: окно → команды → фокус → снимок
python3 tests/test_agent_flow.py    # то же через настоящий MCP-протокол
python3 tests/test_mcp_stdio.py     # протокол MCP: список тулов и вызовы
```

В CI (`.github/workflows/ci.yml`) гоняются линтер, юнит-тесты и проверка импорта — живые тесты в контейнере без композитора не запустятся.

Лицензия

MIT — см. LICENSE.